

数字电路设计第 10 章

I/O 系统（自学）

考试重要性：★（自学内容，不考）

中南大学计算机学院

基于课堂课件整理，页码引用格式 [p.X] 对应课件页码

使用说明：本章为自学内容，不在考试范围内。但作为计算机原理与体系结构课程的重要组成部分，I/O 系统知识对于理解完整的计算机系统架构、尤其是嵌入式系统开发有重要意义。建议在完成考试复习后翻阅。

目录

1 I/O 接口概述 (I/O Interface Overview)	2
2 直接程序控制方式 (Direct Program Control)	4
3 程序中断方式 (Program Interrupt)	5
4 DMA 方式 (Direct Memory Access)	6
5 总线分类与 AMBA (Bus Classification & AMBA)	8
6 系统总线架构 (System Bus Architecture)	10
7 总线控制与仲裁 (Bus Control & Arbitration)	12
8 总线通信方式 (Bus Communication Modes)	14
9 本章核心概念总结与对比	16
10 自学指导与学习建议	17

1 I/O 接口概述 (I/O Interface Overview)

[p.1-9]

I/O 接口的功能

[p.3]

I/O 接口是连接主机（CPU、内存）与外部设备的桥梁，其核心功能包括：

- 寻址功能：对 CPU 送来的地址码进行译码，选择相应的 I/O 设备
- 输入/输出功能：实现 CPU 与 I/O 设备之间的数据传送
- 控制方式支持：支持主机采取程序查询、中断、DMA 等方式访问外设
- 缓冲与驱动：提供主机和 I/O 设备所需的缓冲、暂存和驱动能力，满足一定的负载要求和时序要求
- 数据格式转换：进行数据类型（串行/并行）、格式等方面的转换
- 辅助功能：联络信号、复位、可编程、错误检测等

I/O 接口的分类

[p.3]

分类依据	类型	说明
按数据传送格式	串行接口	一位一位传送，如 UART、SPI
	并行接口	多位同时传送，如 GPIO 并口
按主机访问控制方式	程序查询接口	CPU 轮询外设状态
	中断接口	外设主动通知 CPU
	DMA 接口	直接存储器访问
	通道控制器	专用 I/O 处理器
	I/O 处理机	更高级的 I/O 管理
按时序控制方式	同步接口	统一时钟控制
	异步接口	握手信号控制

I/O 接口编址方式

[p.3]

这是本章一个核心概念。CPU 如何区分访问的是内存还是 I/O 设备？

	存储器映射 I/O (Memory-Mapped I/O)	端口 I/O (Port-Mapped I/O)
原理	I/O 端口与主存统一编址，使用相同的地址空间	I/O 端口独立编址，有独立的地址空间
指令	使用普通访存指令（如 LDR/STR）访问 I/O	使用专用 I/O 指令（如 IN/OUT）访问 I/O
优点	指令丰富，编程灵活；无需专用 I/O 指令	不占用内存地址空间；I/O 地址线少，译码简单
缺点	占用内存地址空间	需要专用 I/O 指令和额外控制信号
典型代表	ARM 架构（如 Raspberry Pi）	x86 架构

I/O 接口的基本组成

[p.4]

I/O 接口的典型结构包括：

- 地址译码逻辑：识别 CPU 发出的 I/O 地址，选中对应外设
- 数据缓冲寄存器：暂存输入输出数据（并行/串行转换）
- 命令/状态寄存器：存放 CPU 发来的控制命令，或供 CPU 查询的设备状态
- 控制电路：产生联络信号、中断请求、DMA 请求等控制时序

外围设备的速度分级

[p.5]

外设种类繁多，速度差异巨大，需要不同的定时策略：

设备类型	速度等级	定时策略
极慢设备（简单 LED、开关）	人类操作级（ms~s）	无需询问，CPU 直接发送/接收数据
慢/中速设备（串口、键盘）	Kbps~Mbps 级	采用异步定时（握手应答）
高速设备（硬盘、显卡）	Mbps~Gbps 级	采用同步定时（统一时钟）

2 直接程序控制方式 (Direct Program Control)

[p.10–13]

特点

[p.10]

CPU 直接通过 **I/O 指令**对 I/O 接口进行操作访问，主机与外设交换信息的**每一个过程**均在程序中表示出来。两种实现方式如下。

立即程序传送方式（无条件传送）

[p.11–12]

- 无需询问接口的状态，CPU 直接利用 I/O 指令访问相应端口
- 适用于**速度极慢或简单**的外设，任何时候都处于”就绪”状态
- 典型应用：GPIO 控制 LED 闪烁（如 Raspberry Pi 的 RPi.GPIO）

示例：树莓派 LED 闪烁（立即传送） [p.12]

```
GPIO.setup(18, GPIO.OUT)
GPIO.output(18, GPIO.HIGH) // 立即点亮，不查询状态
time.sleep(1.0)
GPIO.output(18, GPIO.LOW) // 立即熄灭
```

程序查询方式（条件传送）

[p.13]

- CPU 在程序中**循环查询**接口的状态寄存器
- 若接口**未准备好**（如数据未到达），CPU 原地等待
- 若接口**已就绪**，CPU 执行 I/O 指令进行数据传送
- 缺点：CPU 大量时间浪费在查询等待上，效率低

三种程序控制方式对比：

方式	是否查询状态	适用设备	CPU 效率
立即程序传送	否	极慢简单设备	低（但外设不耗时）
程序查询方式	是（循环等待）	慢/中速设备	低（循环查询浪费 CPU）
程序中断方式	不用查询（外设通知）	各类外设	高（CPU 并行处理）

3 程序中断方式 (Program Interrupt)

[p.14–18]

基本概念

[p.14–15]

中断方式下，外设主动发出请求，CPU 接到请求后：

1. 暂停原有程序的执行
2. 转去执行中断处理程序
3. 处理完毕后恢复原程序的继续执行

与程序查询的本质区别：程序查询是 CPU 主动轮询（CPU 是主动方），中断是外设主动通知（外设是主动方）。

单级中断 (Single-Level Interrupt)

[p.16]

- 所有中断源处于同一优先级
- 离 CPU 越近的中断源，优先级越高（物理位置决定）
- 中断不能被打断：执行中的中断服务程序结束前，不能响应其他中断请求

多级中断 (Multi-Level Interrupt)

[p.18]

- 中断事件分为多级，每一级有一个中断优先权
- 高优先级可打断低优先级中断服务程序的执行（中断嵌套）
- 两种结构：
 - 一维多级中断：优先级沿一根线排列
 - 二维多级中断：多根优先级线，每根线上又有多个中断源

中断源的识别与中断向量

[p.17]

当多个中断源同时发出请求时，需要中断判优来确定优先响应哪个请求。

- 中断请求 (IR)：外设发出的中断请求信号
- 中断选中 (IS)：根据优先级判定，选中某个请求
- 中断响应 (INTA)：CPU 发出的中断响应信号
- 中断排队 (INTI)：多个请求按优先级排队
- 被选中的 IR 信号经编码电路产生中断向量（如 001010），CPU 据此跳转到对应的中断服务程序入口

4 DMA 方式 (Direct Memory Access)

[p.19–24]

定义与基本思想

[p.19]

DMA (Direct Memory Access, 直接存储器访问) 是为了在主存储器与 I/O 设备之间高速交换批量数据而设置的。

基本思想：通过硬件控制实现主存与 I/O 设备间的直接数据传送，在传送过程中无需 CPU 干预。数据传送在 DMA 控制器控制下进行。

优点：速度快，有利于充分发挥 CPU 的效率（CPU 与外设并行工作）。

DMA 控制器的基本构成

[p.20]

- **地址寄存器 (AR)：**存放本次传送数据的主存地址
- **字计数器 (WC)：**统计传送数据的个数（字节/字数）
- **控制/状态寄存器：**确定数据传送方向（读/写）、操作方式等
- **数据缓冲寄存器：**暂存传送的数据
- **总线请求/响应逻辑：**向 CPU 发出 HOLD 总线请求，接收 HLDA 响应信号

DMA 传送过程

[p.21–22]

一个完整的 DMA 数据块传送分为三个阶段：

阶段	执行者	操作
1. 初始化	CPU	通过程序给 DMA 控制器预置初值：传送方向、主存起始地址、传送字数
2. 数据交换	DMA 控制器	DMA 接管总线控制权，控制外设与主存之间的数据交换。每传送一个数据，地址增 1、字数减 1，直至字数 = 0
3. 后处理	CPU	DMA 操作结束时产生中断，CPU 执行后处理（检查数据、决定是否继续传送或停止）

DMA 传送一个数据的详细流程

[p.21]

1. 外设发出 DMA 请求 → DMA 控制器向 CPU 发出**总线请求 (HOLD)**
2. CPU 响应，发出**总线响应 (HLDA)** → DMA 控制器接管总线控制，进入 DMA 操作周期
3. DMA 控制器根据当前主存地址，确定数据传送方向，发出读写控制信号，执行数据传送
4. 传送完成后，DMA 控制器向 CPU 报告操作结束

DMA 的特点

[p.23]

- 使主存与 CPU 的固定联系脱钩：主存既可以被 CPU 访问，又可被外设访问
- 主存地址的确定、传送数据的计数等都由**硬件电路**直接实现
- 主存中需开辟专用缓冲区，及时供给和接收外设数据
- DMA 和 CPU **并行工作**，提高系统效率
- 传送开始前通过程序进行**预处理**（初始化），结束后通过中断方式进行**后处理**

中断与 DMA 的核心区别

[p.24]

高频考点

对比维度	程序中断方式	DMA 方式
本质	程序的切换	除了开始和结尾，不占用 CPU 任何资源
响应时机	只能在每条指令 执行完毕 时响应	可在每个 机器周期结束 时响应
CPU 干预	传送全过程需 CPU 干预	不需要 CPU 干预
数据传输速率	较慢	非常高 ，适合高速外设成组数据传送
优先级	低于 DMA 请求	高于 中断请求
功能范围	具有对 异常事件 的处理能力	仅限于 完成传送数据块的 I/O 操作
适用场景	低速设备、字符/状态传送、异常处理	高速设备、成组数据传送（磁盘、网络）

5 总线分类与 AMBA (Bus Classification & AMBA)

[p.25–27]

总线的分类

[p.25]

分类依据	类型	说明
按传输方式	串行传输总线	数据一位一位传送，如 SPI、I ² C
	并行传输总线	数据多位同时传送，如 PCI
按使用范围	计算机总线	PC 内部连接总线
	测控总线	工业测控系统
	网络通讯总线	计算机网络（如 Ethernet）
按连接部件	片内总线	芯片内部的总线（如 ARM AMBA）
	系统总线	CPU、内存、I/O 设备各大部件之间的传输线
	通信总线	计算机与计算机/外设之间的通信

系统总线的三类信号

[p.25]

- **数据总线 (Data Bus):** 双向，传送数据。宽度决定了每次传送的位数（如 32/64 位）
- **地址总线 (Address Bus):** 单向（CPU 发出），传送地址。宽度决定了寻址范围
- **控制总线 (Control Bus):** 传送控制信号（读/写、中断、DMA 请求/响应等）

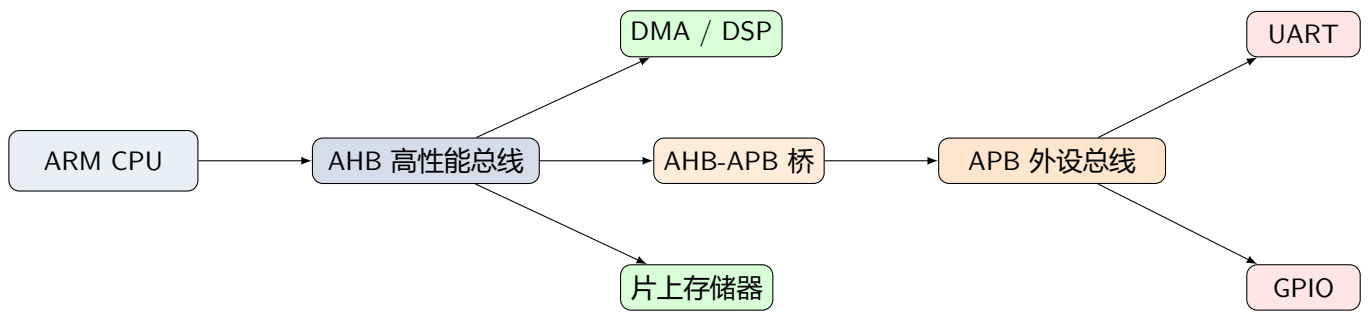
AMBA 片内总线 (Advanced Microcontroller Bus Architecture)

[p.26–27]

AMBA 是 ARM 公司 1996 年推出的**开源片内总线标准**，用于连接芯片上的各组件。它不是芯片与外设之间的接口，而是 **ARM 内核与芯片上其他元件进行通信的接口**。

AMBA 2.0 定义的三组总线：

总线	全称	定位	用途
AHB	AMBA High-Performance Bus	高性能总线	用于高性能、高数据吞吐部件之间的连接，如 CPU、DMA、DSP
ASB	AMBA System Bus	系统总线	处理器与外设互连，将被 AHB 取代
APB	AMBA Peripheral Bus	外设总线	为系统的低速外部设备提供低功耗的简易互连，如 UART、GPIO、Timer



图：AMBA 典型架构——AHB 连接高速部件，APB 连接低速外设

6 系统总线架构 (System Bus Architecture)

[p.28–37]

单总线结构 (Single Bus)

[p.28]

- 只有一条总线（系统总线），把 CPU、内存、I/O 设备全部连接起来
- 优点：结构简单，成本低，易于扩展
- 缺点：所有设备共享带宽，成为性能瓶颈；CPU 和内存通信与 I/O 通信相互争抢总线

双总线结构 (Dual Bus)

[p.29–31]

A. 面向 CPU 的双总线结构： [p.29]

- 新增一条存储总线 (M 总线)，专门连接 CPU 和内存（高速、频繁通信）
- 原有系统总线改为 I/O 总线，连接 CPU 和其他 I/O 设备
- 优点：CPU-内存通信不再与 I/O 设备争抢带宽

B. 面向存储器的双总线结构： [p.30]

- 主存一边通过存储总线与 CPU 通信，一边也连接在系统总线上与其他设备通信
- 允许 I/O 设备绕过 CPU 直接与主存交换数据

C. 通道方式的 I/O 结构： [p.31]

- I/O 设备统一通过通道来管理
- 通道是一种特殊的处理器，专门负责 I/O 设备与其他设备的通信

三总线结构 (Triple Bus)

[p.32–33]

- 在双总线基础上新增一条 DMA 总线，让高速 I/O 设备直接与主存通信
- 问题：低速 I/O 设备仍需经过 CPU 访问主存，占用 CPU 资源
- 另一种三总线结构：CPU 只与 Cache 通信；Cache 缺失时从主存取数据；所有 I/O 设备通过扩展总线和系统总线与主存通信（但不区分高速/低速）

四总线结构 (Quad Bus)

[p.34]

- 在三总线基础上新增一条高速总线，连接高速 I/O 设备
- 高速 I/O 设备距离 CPU 和主存更近，通信更快
- 扩展总线连接低速 I/O 设备
- 实现了高速/低速设备的分离，性能最优

PCI 总线结构 (现代 PC)

[p.35–37]

- CPU 通过系统总线（前端总线 FSB）与主存连接
- 系统总线上连接了 **PCI 桥**，通过 PCI 桥可连接 PCI 总线
- PCI 总线下可连接符合 PCI 接口的 I/O 设备（如网卡、声卡）
- **多层 PCI 结构**：PCI 桥下再连接 PCI 桥 → 形成 PCI 总线树，可一直向下扩展
- **南北桥架构**：北桥（North Bridge）连接 CPU、内存、高速 PCIe；南桥（South Bridge）连接低速外设（USB、SATA、音频等）

7 总线控制与仲裁 (Bus Control & Arbitration)

[p.38–42]

主设备与从设备

[p.38]

- **主设备 (Master):** 对总线有控制权，可以启动数据传送
- **从设备 (Slave):** 只能响应主设备发来的总线命令，对总线没有控制权
- 总线上的信息传送由**主设备启动**，只有获得总线控制权的主设备才能开始传送数据
- 当多个主设备同时申请总线使用权时，需要**总线仲裁**

集中控制优先权仲裁的三种方式

[p.39–41]

这是本章关于总线仲裁的核心内容。

	链式查询 (Daisy Chain)	计数器定时查询 (Counter Query)	独立请求 (Independent Request)
原理	总线授权信号串行传递，离控制器近的优先	控制器发出设备地址计数，计数匹配的设备获得总线	每个设备有独立请求线和授权线
控制线数	最少（3 根）	中等（需设备地址线）	最多（每个设备 2 根）
响应速度	较慢（串行传递）	中等	最快
优先级灵活性	固定（物理位置决定）	灵活（可改变计数初值）	最灵活（可编程改变）
故障敏感性	非常敏感（一处断，全链失效）	较不敏感	较不敏感
扩展性	容易	较复杂	较复杂
低优先级”饿死”	可能（低优先级难获得请求）	可能	可通过编程避免

总线周期的四个阶段

[p.42]

完成一次总线操作的时间称为**总线周期**，分为以下四个阶段：

阶段	名称	操作
1	申请分配	主模块提出申请，经仲裁机构决定将总线使用权授予某一申请者
2	寻址	取得使用权的主模块发出从模块地址及命令，启动从模块
3	传输	主模块和从模块进行数据交换（源模块发 → 目的模块收）
4	结束	主模块有关信息从总线上撤出，让出总线使用权

8 总线通信方式 (Bus Communication Modes)

[p.43–49]

四种通信方式对比

[p.43, 49]

	同步通信	异步通信	半同步通信
时钟	统一的公共时钟	无公共时钟	统一时钟 + 等待信号
协调机制	统一时标	握手/应答 (Handshaking)	插入等待周期
速度	快（无握手开销）	慢（握手开销）	中等
适用场景	总线长度短、存取时间统一	速度差异大的设备	快慢设备混合
典型协议	SPI	UART（串行异步）、 RS-232	部分 SRAM 接口

异步通信的握手方式

[p.45]

异步通信通过 **Request**（请求）和 **Acknowledge**（应答）两条握手信号线协调数据传送。

- **不互锁**：主模块发 Request 后不等 Acknowledge 就继续；从模块发 Acknowledge 后也不等
- **半互锁**：主模块发 Request 后等 Acknowledge，但从模块发 Acknowledge 后不等确认
- **全互锁**：双方都互相等待对方的确认信号，最可靠

异步串行通信

[p.46]

异步串行通信的**字符格式**：

- **1 个起始位**（低电平）：标志字符开始
- **5~8 个数据位**：实际传送的数据
- **1 个奇偶校验位**：用于错误检测（可选）
- **1、1.5 或 2 个停止位**（高电平）：标志字符结束
- 起始位后紧跟字符的**最低位**（LSB 优先）

波特率 (Baud Rate)：单位时间内传送二进制数据的位数，单位用 **bps**（位/秒）表示，记作波特。常见波特率：9600、19200、115200 bps。

SPI 和 UART 简介

[p.44, 47]

	SPI	UART
全称	Serial Peripheral Interface	Universal Asynchronous Receiver/Transmitter
同步/异步	同步串行协议	异步串行协议
时钟	有 SCK 时钟线	无时钟线（双方约定波特率）
信号线	SCK, MOSI, MISO 三线	TX, RX 两线
特点	简单、快速、全双工	常用于 RS-232、RS-485
速度	较快（有同步时钟）	较慢（异步，需起始/停止位开销）
典型应用	传感器、LCD、SD 卡	串口调试、GPS 模块、蓝牙模块

分离式通信

[p.49]

- 将一个传输周期分解为两个子周期
- 子周期 1：主模块 A 发送命令、地址及自身编号后立即放弃总线使用权
- 子周期 2：从模块 B 准备好数据后，由 B 申请总线使用权，将数据及 A 的编号发回
- 优点：总线利用率高（不等待期间其他模块可使用总线）
- 缺点：控制复杂，微型计算机系统中很少采用

9 本章核心概念总结与对比

三种数据传送方式终极对比

	程序查询	程序中断	DMA
发起方	CPU（主动轮询）	外设（主动通知）	外设/DMA 控制器
CPU 干预	全过程干预	传送时干预	仅开始/结束干预
传送单位	字/字节	字/字节	数据块
速度	慢	中	快
CPU 效率	低	中	高
适用	慢速设备	慢/中速设备	高速批量数据
硬件复杂度	简单	中等	复杂

存储器映射 I/O vs 端口 I/O

	存储器映射 I/O (Memory-Mapped)	端口 I/O (Port-Mapped)
地址空间	I/O 与主存共享	I/O 独立编址
使用指令	普通访存指令 (LDR/STR)	专用 I/O 指令 (IN/OUT)
典型架构	ARM	x86
优点	指令集统一、编程灵活	不占内存地址，译码简单

总线分类一览

按连接部件	典型实例	说明
片内总线	AMBA (AHB/APB)	芯片内部
系统总线	数据 + 地址 + 控制总线	连接 CPU、内存、I/O
通信总线	SPI, UART, USB, PCIe	连接外设/外部设备

10 自学指导与学习建议

本章学习要点

虽然本章不在考试范围内，但以下知识点对于理解完整的计算机系统架构具有重要意义：

- **存储器映射 I/O vs 端口 I/O**——理解 ARM vs x86 设计哲学差异的关键，在嵌入式开发中经常遇到
- **中断与 DMA 的区别**——这是操作系统和驱动程序开发的基础知识
- **总线仲裁方式**——链式查询、计数器查询、独立请求的工程权衡体现了设计中的速度/复杂度/成本折中
- **AMBA 总线**——几乎所有 ARM SoC 都使用 AMBA，是嵌入式/移动芯片设计的基础
- **SPI/UART**——实际做嵌入式项目时最常用的通信协议

建议学习路径

1. 先理解 I/O 接口的基本功能、分类、编址方式（两种编址的区别是核心）
2. 对比学习三种数据传送方式：程序查询 → 中断 → DMA（复杂度递增，效率也递增）
3. 重点理解中断和 DMA 的核心区别表（p.24）
4. 了解总线分类和 AMBA 三总线结构（AHB/ASB/APB）
5. 对照现代 PC 的总线架构（四总线 +PCI），理解总线结构演变的原因
6. 浏览 SPI/UART 等实际接口，为嵌入式开发积累

本章为自学内容，不纳入考试范围。学习时间建议：1~2 小时。